

● SEGURANÇA EM DADOS

Administração Segura de Bancos de Dados

Linguagem SQL para Controle de Acesso — **DCL na prática**, com criação de usuários, perfis e gestão de permissões.

OBJETIVOS DA AULA

- 1 Compreender o papel da **administração segura** de bancos de dados
- 2 Conhecer **estratégias de proteção** dos dados
- 3 Gerenciar permissões utilizando **DCL (GRANT e REVOKE)**
- 4 Criar **perfis de usuários** e definir níveis de privilégios

- DCL
- GRANT
- REVOKE

Um banco de dados sem administração segura é uma porta aberta para vazamentos e perdas

Dados são o ativo mais valioso das organizações. Vazamentos geram **prejuízos financeiros**, dano à reputação e sanções legais como **LGPD** e **GDPR**. A maioria dos incidentes com dados envolve privilégios mal configurados e falhas de backup — ambos **100% evitáveis** com boa administração.

A TRÍADE DA SEGURANÇA — CID

Confidencialidade

Apenas pessoas autorizadas acessam os dados.

Integridade

Os dados permanecem corretos e consistentes.

Disponibilidade

Os dados estão acessíveis quando necessário.

AMEAÇAS COMUNS

- **Acesso não autorizado** — contas compartilhadas e senhas fracas abrem brechas diretas.
- **Injeção de SQL** — comandos maliciosos exploram falhas em aplicações.
- **Perda de dados** — falhas de hardware, erros humanos e backups ausentes ou corrompidos.
- **Privilégios excessivos** — usuários com mais permissões do que precisam ("menor privilégio" ignorado).

DBA — DATABASE ADMINISTRATOR

O profissional guardião da infraestrutura de dados: garante a tríade **CID**, administra permissões, backups, desempenho e atualizações do SGBD.

Segurança começa na estrutura: um banco bem organizado é mais fácil de proteger

A hierarquia do SGBD organiza dados em camadas — cada camada é um ponto de controle de acesso. **Versões usadas nos exemplos: MySQL e PostgreSQL.**

HIERARQUIA DE ORGANIZAÇÃO

- 1 **Servidor de banco de dados** — instância onde o SGBD roda (MySQL, PostgreSQL, Oracle, SQL Server).
- 2 **Usuários e esquemas** — cada usuário possui um esquema que organiza seus objetos.
- 3 **Banco de dados** — contém tabelas, views, stored procedures, triggers e índices.
- 4 **Tabelas** — colunas (atributos) e linhas (registros), com chaves primárias e estrangeiras para integridade.
- 5 **Objetos adicionais** — views restringem colunas sensíveis; procedures encapsulam lógica.

COMO FAZER — EXEMPLO SQL

PASSO 1 · CRIAR O BANCO E A TABELA

```
CREATE DATABASE empresa_segura;  
USE empresa_segura;  
CREATE TABLE funcionarios (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  nome VARCHAR(100) NOT NULL,  
  salario DECIMAL(10,2) NOT NULL,  
  departamento VARCHAR(50)  
);
```

PASSO 2 · ESCONDER DADOS SENSÍVEIS COM UMA VIEW

```
CREATE VIEW vw_funcionarios_publico AS  
SELECT id, nome, departamento  
FROM funcionarios;  
-- a coluna salario fica oculta para quem acessa a view
```



Chave de integridade: definir chaves primárias e estrangeiras garante que os relacionamentos entre tabelas permaneçam consistentes.

O DBA é o guardião da infraestrutura de dados: quatro frentes de atuação

A administração de banco de dados combina rotinas diárias, semanais e mensais. O **DBA (Database Administrator)** garante segurança, desempenho, disponibilidade e integridade de forma contínua.

RESPONSABILIDADES DO DBA

Segurança

Criar usuários, definir permissões, auditar acessos e aplicar criptografia.



Desempenho

Criar índices, otimizar queries e monitorar o uso de recursos.



Disponibilidade

Configurar backups, planos de recuperação de desastres (DR) e alta disponibilidade.



Integridade

Manter consistência dos dados, aplicar regras de negócio e atualizar patches do SGBD.

CHECKLIST — TAREFAS ROTINEIRAS

- 1 **Verificar backups** e testar a restauração periodicamente.
- 2 **Auditar logs** de acesso e ações suspeitas.

- 3 **Revisar permissões** e remover privilégios órfãos.
- 4 **Monitorar disco** e desempenho das queries.
- 5 **Aplicar atualizações** de segurança do SGBD.

O kit do DBA: ferramentas certas tornam a administração repetível e auditável

Ferramentas administrativas organizam o trabalho do DBA em quatro frentes — **gestão visual, proteção de dados, monitoramento e auditoria**. Dominar ao menos uma ferramenta de cada categoria é o padrão mínimo profissional.

CATEGORIA	EXEMPLOS	USO PRINCIPAL
Gerenciadores gráficos	DBeaver, MySQL Workbench, pgAdmin, SSMS	Gerenciar conexões, esquemas e usuários visualmente
Backup e restauração	mysqldump, pg_dump, Oracle RMAN	Copiar e restaurar dados com consistência
Monitoramento	Prometheus + Grafana, pgbadger, Performance Monitor	Identificar gargalos e uso anormal de recursos
Auditoria e logs	Auditing nativo dos SGBDs, plugins de auditoria	Rastrear quem fez o quê e quando

COMO FAZER — BACKUP E RESTAURAÇÃO NA PRÁTICA

MYSQL · PASSO A PASSO
1. Fazer backup completo do banco
mysqldump -u root -p \
--databases empresa_segura \
> backup.sql

2. Restaurar o backup
mysql -u root -p empresa_segura \
< backup.sql

POSTGRESQL · PASSO A PASSO
1. Fazer backup do banco
pg_dump -U postgres \
empresa_segura > backup.sql

2. Restaurar o backup
psql -U postgres \
empresa_segura < backup.sql

Segurança eficaz combina várias camadas de defesa — não apenas uma medida

Seis estratégias fundamentais, com o **passo a passo de aplicação** de cada uma. Aplicadas em conjunto, formam a **defesa em profundidade**.

1 Menor privilégio

Conceder apenas as permissões mínimas necessárias para cada função.

Como fazer: mapear funções → listar permissões necessárias → conceder via GRANT específico → auditar mensalmente

2 Segregação de funções

Separar quem desenvolve, quem administra e quem apenas consulta.

Como fazer: criar perfis (roles) distintos: dev · analista · admin · app_service

3 Backups com teste de restauração

Backup não testado é backup inexistente — restauração é a única prova real.

Como fazer: agendar backup diário → restaurar em ambiente de teste → validar integridade

4 Criptografia

Proteger dados em repouso (TDE) e em trânsito (SSL/TLS).

Como fazer: ativar conexão SSL → ativar TDE no SGBD para tablespaces sensíveis

5 Auditoria e logs

Registrar quem acessou, quem alterou o quê e quando — rastreabilidade total.

Como fazer: ativar auditing nativo do SGBD → monitorar tabelas críticas → revisar logs periodicamente

6 Atualização e hardening

Reduzir a superfície de ataque do ambiente de banco de dados.

Como fazer: manter SGBD atualizado → alterar portas padrão → desabilitar contas inúteis

Defesa em profundidade: se uma camada falhar, as demais continuam protegendo os dados — por isso nenhuma estratégia sozinha é suficiente.

DCL é a parte do SQL dedicada a controlar quem pode fazer o quê

O SQL é organizado em famílias de comandos. O **DCL (Data Control Language)** concentra-se exclusivamente em permissões — é o comando central da segurança. Fonte: documentação oficial MySQL / PostgreSQL.

ONDE O DCL SE ENCAIXA

FAMÍLIA	FUNÇÃO	COMANDOS
DDL	Definir estruturas	CREATE, ALTER, DROP
DML	Manipular dados	SELECT, INSERT, UPDATE, DELETE
DCL	Controlar acesso	GRANT, REVOKE
TCL	Transações	COMMIT, ROLLBACK, SAVEPOINT

🔑 **GRANT concede / REVOKE revoga:** são comandos complementares e simétricos — tudo que um faz, o outro desfaz.

CONCEITOS-CHAVE

- **Privilégio**
Permissão para executar uma ação sobre o banco: **SELECT, INSERT, UPDATE, DELETE** (objeto) ou **CREATE USER, ALTER DATABASE** (sistema).
- **Perfis (roles)**
Grupos de privilégios atribuídos a usuários — **gerencie perfis** em vez de usuários individuais: uma mudança no role se propaga a todos.

ANATOMIA · CONCEDER

GRANT <privilegios>
ON <objeto>
TO <usuário>;

ANATOMIA · REVOGAR

REVOKE <privilegios>
ON <objeto>
FROM <usuário>;

GRANT concede e REVOKE revoga: comandos complementares que controlam quem pode fazer o quê

Dominar a sintaxe completa evita brechas de segurança: identifique **privilegio**, **objeto** e **usuário** antes de executar — e sempre verifique o resultado.

● GRANT — COMO USAR

- 1 Identificar o **privilegio**: sistema (CREATE USER, ALTER DATABASE) ou de objeto (SELECT, INSERT, UPDATE, DELETE, ALL PRIVILEGES).
- 2 Identificar o **objeto**: tabela, view, banco ou *.* para todo o SGBD.
- 3 Identificar o **usuário ou role** de destino.
- 4 (Opcional) WITH GRANT OPTION — permite conceder a terceiros: **usar com extrema cautela**.

EXEMPLOS PRÁTICOS

```
GRANT SELECT ON funcionarios TO analista_dados;
GRANT SELECT, INSERT ON vw_folha TO rh_operacao;
GRANT ALL PRIVILEGES ON empresa_segura.* TO admin_db;
GRANT CREATE USER ON *.* TO admin_seguranca;
```

● REVOKE — COMO USAR

- 1 Identificar o **privilegio a ser retirado** — sem WITH GRANT OPTION, retire também o GRANT OPTION separadamente, se houver.
- 2 Identificar o **objeto e o usuário** de destino.
- 3 Executar e **verificar** a revogação consultando as permissões atuais (SHOW GRANTS).

EXEMPLOS PRÁTICOS

```
REVOKE INSERT ON funcionarios FROM rh_operacao;
REVOKE ALL PRIVILEGES ON *.* FROM usuario_exfunc;
```

⚠ **Alerta:** revogar um privilégio **não exclui o usuário** — para remover o acesso total, use DROP USER.

O fluxo completo de controle de acesso em 6 passos executáveis

Crie usuários e perfis do zero, do comando `CREATE USER` ao papel padrão — cada passo pode ser executado diretamente no MySQL. **Exemplo: equipe com analista, operação e administração.**

1 Criar os usuários

```
CREATE USER 'ana'@'localhost'  
  IDENTIFIED BY 'SenhaForte#2026';  
CREATE USER 'bruno'@'localhost'  
  IDENTIFIED BY 'SenhaForte#2026';  
CREATE USER 'app_servico'@'%'  
  IDENTIFIED BY 'SenhaForte#2026';  
-- '%' = aceita qualquer origem
```

2 Criar os perfis (roles)

```
CREATE ROLE 'rol_leitura';  
CREATE ROLE 'rol_operacao';  
CREATE ROLE 'rol_admin';
```

3 Conceder privilégios aos perfis

```
GRANT SELECT ON funcionarios  
  TO 'rol_leitura';  
GRANT SELECT, INSERT, UPDATE ON funcionarios  
  TO 'rol_operacao';  
GRANT ALL PRIVILEGES ON *.*  
  TO 'rol_admin' WITH GRANT OPTION;
```

4 Atribuir perfis aos usuários

```
GRANT 'rol_leitura'  
  TO 'ana'@'localhost';  
GRANT 'rol_operacao'  
  TO 'bruno'@'localhost';  
GRANT 'rol_admin'  
  TO 'app_servico'@'%';
```

5 Forçar aplicação das permissões

```
FLUSH PRIVILEGES;  
-- recarrega as tabelas de  
-- privilégios na memória
```

6 Definir perfil padrão (MySQL 8+)

```
SET DEFAULT ROLE 'rol_leitura'  
  TO 'ana'@'localhost';  
SET DEFAULT ROLE 'rol_operacao'  
  TO 'bruno'@'localhost';
```



Por que perfis? Reutilização e manutenção centralizada — mudou um privilégio no role, todos os usuários herdam a mudança automaticamente.

Administrar é contínuo: é preciso inspecionar, corrigir e revogar permissões

Um colaborador trocou do **RH para o financeiro**. O que antes era certo — **rol_operacao** no perfil — agora é um risco. Veja como corrigir passo a passo.

COMO VERIFICAR PERMISSÕES

PRIVILÉGIOS DE UM USUÁRIO

```
SHOW GRANTS FOR 'ana'@'localhost';
```

PRIVILÉGIOS DO PRÓPRIO USUÁRIO

```
SHOW GRANTS FOR CURRENT_USER();
```

Q **Regra de ouro:** após cada GRANT ou REVOKE, revalide com SHOW GRANTS — nunca confie na memória.

● CENÁRIO DE CORREÇÃO — BRUNO MUDOU DO RH PARA O FINANCEIRO

- 1 Verificar permissões atuais: **SHOW GRANTS FOR 'bruno'@'localhost';**
- 2 Revogar o perfil antigo: **REVOKE 'rol_operacao' FROM 'bruno'@'localhost';**
- 3 Se o role não é padrão, revogar privilégios diretos: **REVOKE INSERT ON funcionarios FROM 'bruno'@'localhost';**
- 4 Atribuir o novo perfil: **GRANT 'rol_leitura' TO 'bruno'@'localhost';**
- 5 Atualizar perfil padrão: **SET DEFAULT ROLE 'rol_leitura' TO 'bruno'@'localhost';**
- 6 Aplicar e revalidar: **FLUSH PRIVILEGES;** + **SHOW GRANTS;**

TABELA DE DECISÃO

SITUAÇÃO	AÇÃO CORRETA
Usuário precisa de nova função	Atribuir novo role + revogar o antigo
Usuário saiu da empresa	REVOKE todos os privilégios + DROP USER
Privilégio concedido em excesso	REVOKE do privilégio excessivo imediatamente
Colaborador temporário (estágio)	Só SELECT sobre views — nunca sobre tabelas brutas

Mão na massa: "Segure o vazamento" — corrija um incidente real com GRANT e REVOKE



A empresa **DataCorp** teve um incidente de segurança: um estagiário acessou a coluna de salários e um ex-funcionário ainda consegue fazer login no banco de dados. **Sua missão é corrigir o ambiente aplicando as boas práticas vistas em aula.**

DURAÇÃO TOTAL: 45 MINUTOS

1 Preparar o ambiente

15 min

Monte o cenário de trabalho no SGBD.

- Criar o banco **datacorp** e a tabela `colaboradores(id, nome, cargo, salario)`.
- Criar a view **vw_colaboradores_publico** sem a coluna `salario`.

```
CREATE VIEW vw_colaboradores_publico AS
SELECT id, nome, cargo
FROM colaboradores;
```

2 Simular o cenário errado

10 min

Reproduzir, propositalmente, os erros de permissão que causaram o incidente.

- Criar os usuários **estagiario** e **exfunc**.
- Conceder `GRANT ALL PRIVILEGES` a ambos — **incorreto, de propósito**.

```
CREATE USER 'estagiario'@'localhost'
IDENTIFIED BY 'SenhaForte#2026';
GRANT ALL PRIVILEGES ON datacorp.* TO
'estagiario'@'localhost';
```

3 Corrigir com boas práticas

20 min

Resolver o incidente do início ao fim.

- Criar roles **rol_estagio** (`SELECT` na view apenas) e **rol_analista** (`SELECT/INSERT` nas tabelas).
- Revogar `ALL PRIVILEGES` de ambos e atribuir o role correto ao estagiário.
- Encerrar o ex-funcionário: `REVOKE ALL + DROP USER`.
- Testar: `SELECT` em **salario** deve falhar; na view deve funcionar.



Critério de sucesso: o estagiário só enxerga nome, cargo e departamento · o ex-funcionário não consegue mais fazer login · todos os usuários operam por meio de roles.

Revisar as soluções em grupo consolida o aprendizado — segurança é processo contínuo

A correção coletiva transforma o erro em aprendizado. Conduza em **5 movimentos** e feche com o quiz de fixação e a síntese final.

COMO CONDUZIR A CORREÇÃO COLETIVA

- 1 Coletar 5 min**
Pedir que os alunos compartilhem suas soluções via tela ou código.
- 2 Validar 10 min**
Testar ao vivo cada comando — quem conseguiu o **REVOKE** funcionando? A view bloqueou o salário?
- 3 Discutir erros comuns 10 min**
FLUSH PRIVILEGES esquecido, privilégio direto vs. role, WITH GRANT OPTION abusado, REVOKE sem DROP USER.
- 4 Comparar 10 min**
Mostrar a solução-modelo e contrastar com as abordagens alternativas válidas.
- 5 Fixar 5 min**
Quiz rápido oral ou formulário com 5 perguntas.

QUIZ DE FIXAÇÃO

- 1** Qual comando **concede** privilégios?
- 2** E qual comando **revoga**?
- 3** Por que usar **roles** em vez de conceder privilégios diretamente?
- 4** REVOKE ALL remove o usuário do SGBD?
- 5** Qual **princípio** guia a concessão mínima de permissões?

1 → GRANT · 2 → REVOKE · 3 → manutenção centralizada · 4 → Não, é preciso DROP USER
· 5 → Menor privilégio

SÍNTESE DA AULA



Organizar bem o banco → administrar com **estratégia em camadas** → controlar acesso com **DCL** → revisar permissões continuamente.

PRÓXIMOS PASSOS

Estudar auditoria de banco de dados e criptografia (TDE / SSL).